

# Module 1 — Authentication, Registration & SP Invoicing

## — Viva File Guide (Dinuka)

---

Everything you own, ranked by how likely you are to be asked to open it and explain it. Backend is ~1,100 lines total, frontend ~1,300. It is genuinely small enough to know cold.

---

### Tier 1 — Know these line by line (5 files, ~800 lines)

---

If the panel says "show me where X happens", it is almost certainly in one of these.

#### 1. `server/src/modules/auth/auth.service.js` (240 lines)

The single most important file you own. Contains:

- `registerClient()` — uniqueness checks, hash, unique token, then a **transaction** writing `users` + `client_profiles` + `client_locations` together.
- `login()` — verify password → block `DEACTIVATED` → block active ban → create session → sign JWT.
- `logout()` — revokes exactly this session row.
- `forgotPassword()` — **returns silently when the email is unknown** (no account enumeration),

otherwise stores a *hashed* token and emails the raw one.

- `resetPassword()` — transaction: update password + mark the token used, in one commit.
- `generateUniqueToken()`, `signSession()`, `createSession()`, `toPublicUser()`.

#### 2. `server/src/modules/registration/registration.service.js` (255 lines)

Service Provider registration, and the most interesting business logic in your module.

- `registerProvider()` — 4 required documents, then a transaction writing `users` + `service_provider_profiles` + `provider_service_categories` + `sp_verification_applications` + 4 × `sp_verification_documents`.
- `resolveReapplicant()` — the rejected-provider reapply rule. Same username **and** same email

**and** role `SERVICE_PROVIDER` **and** status `REJECTED` **and** the correct current password. Anything else is a plain 409.

- `MAX_VERIFICATION_ATTEMPTS = 3` — three rejections and that identity is permanently done.
- `reapplyAsProvider()` — reuses the account graph, adds a **new attempt** on top of the intact

rejected one. Old attempts are never deleted.

#### 3. `server/src/modules/invoices/invoice.service.js` (163 lines)

- `loadCompletedBookingForProvider()` — the guard: booking exists, belongs to this provider,

status is `COMPLETED`.

- `generateInvoice()` — Card amount is **read from Visal's payment record**, Cash amount is typed

by the provider. Writes the PDF first, inserts the row second, and **deletes the file again if the insert fails** — no orphan PDFs.

- `getInvoiceDownload()` — three legitimate readers: owning provider, owning client, System Admin.
- `getInvoiceFormData()` — autofill payload; `amountEditable` is `false` for Card.

#### 4. `server/src/modules/invoices/invoicePdf.js` (153 lines)

The actual PDF, drawn with **pdfkit**. Green header band, `INV-000123` numbering, a self-sizing two-column table (row heights computed before borders are drawn so a long job description cannot clip), total line, closing line, logo.

#### 5. `database/schema.sql` — the `users` table (around line 1470)

```
CONSTRAINT chk_token_format CHECK (  
  user_token ~ '^[A-Za-z0-9]{6}$' AND user_token ~ '[A-Z]'  
  AND user_token ~ '[a-z]' AND user_token ~ '[0-9]')
```

The 6-character token format is enforced by the database, not just by JavaScript. Also note `username` and `email` are `citext` — case-insensitive by column type, so `Dinuka` and `dinuka` cannot both register.

## Tier 2 — Know what they do and be able to find things in them

| File                                                   | Why it matters                                                                                                                                     |
|--------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>auth/auth.queries.js</code> (142)                | All auth SQL. <code>findSessionValidity()</code> is the one-query live check                                                                       |
| <code>auth/auth.routes.js</code>                       | 6 endpoints; <code>/me</code> and <code>/logout</code> are the only authenticated ones                                                             |
| <code>auth/auth.validation.js</code>                   | Zod: username, Sri Lankan phone, 8+ chars with letter and digit, password match                                                                    |
| <code>registration/registration.queries.js</code> (81) | Provider profile, categories, application, documents                                                                                               |
| <code>invoices/invoice.routes.js</code>                | Mounted at <code>/api/provider/invoices</code> ; the download route uses <code>authorizeRoles('SERVICE_PROVIDER', 'SYSTEM_ADMIN', 'CLIENT')</code> |
| <code>invoices/invoice.queries.js</code>               | 4 queries, including <code>findInvoicesForBookings()</code> — batch lookup that avoids an N+1 on Completed Jobs                                    |
| <code>middleware/authenticate.js</code>                | Verifies the JWT, then <b>chains into</b> <code>checkAccountStatus</code>                                                                          |
| <code>middleware/checkAccountStatus.js</code>          | Re-checks the DB on <b>every</b> request so a ban or logout takes effect immediately                                                               |
| <code>middleware/uploadFiles.js</code>                 | Multer. 5 MB cap, JPG/PNG only (PDF also allowed for the police report), UUID filenames                                                            |
| <code>utils/tokenGenerator.js</code>                   | 6 chars, guaranteed one upper + one lower + one digit, then Fisher-Yates shuffled                                                                  |
| <code>utils/passwordUtils.js</code>                    | bcrypt, <b>12 salt rounds</b>                                                                                                                      |
| <code>emails/email.service.js</code>                   | Resend API. Every send is wrapped in try/catch and only logs on failure                                                                            |

**Your tables:** `users`, `roles`, `auth_sessions`, `password_reset_tokens`, `invoices`. **Tables you write into but do not own:** `client_profiles`, `client_locations` (Tharinsa), `service_provider_profiles`, `provider_service_categories` (Maheli), `sp_verification_applications`, `sp_verification_documents` (Vihas).

## Tier 3 — Frontend (know the flow, not every line)

| File                                                                                                                    | Role                                                                            |
|-------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------|
| <code>features/auth/pages/ProviderRegistrationPage.jsx</code> (170)                                                     | Multi-step provider signup, <code>FormData</code> upload                        |
| <code>features/auth/components/ClientRegistrationForm.jsx</code> (141)                                                  | Client signup + map location                                                    |
| <code>features/auth/components/LoginForm.jsx</code> (86)                                                                | Login + <code>resolveRedirect()</code> — <b>this is "Redirect by Role"</b>      |
| <code>features/auth/pages/RegistrationRolePage.jsx</code> (59)                                                          | Client vs Service Provider choice                                               |
| <code>features/auth/pages/ForgotPasswordPage.jsx</code> / <code>ResetPasswordPage.jsx</code>                            | Password recovery                                                               |
| <code>features/auth/pages/VerificationPendingPage.jsx</code> / <code>ApplicationRejectedPage.jsx</code>                 | Where an unapproved provider lands                                              |
| <code>features/auth/components/ProviderDocumentForm.jsx</code> / <code>TermsModal.jsx</code>                            | 4 documents + T&C acceptance                                                    |
| <code>context/AuthContext.jsx</code> (66)                                                                               | Token storage, <code>login</code> / <code>logout</code> , current user          |
| <code>components/routing/ProtectedRoute.jsx</code> / <code>RoleProtectedRoute.jsx</code> / <code>PublicRoute.jsx</code> | Route-level role gates                                                          |
| <code>features/provider/pages/CreateInvoicePage.jsx</code> (176)                                                        | Invoice form — autofilled, amount locked for Card                               |
| <code>features/provider/components/InvoiceRowAction.jsx</code> (45)                                                     | The Generate/Download control <b>hosted inside Maheli's Completed Jobs page</b> |

### The System Admin "Add User" path — your backend, Vihas's screen

On User Management, the + button opens a menu offering *Client* or *Provider*. Both modals call **your** API functions directly:

| File                                                                     | Calls                                                                          |
|--------------------------------------------------------------------------|--------------------------------------------------------------------------------|
| <code>features/admin/system/components/AddUserMenu.jsx</code> (63)       | Opens one of the two modals                                                    |
| <code>features/admin/system/components/AddClientModal.jsx</code> (150)   | <code>registerClient()</code> → <code>POST /api/auth/register/client</code>    |
| <code>features/admin/system/components/AddProviderModal.jsx</code> (168) | <code>registerProvider()</code> → <code>POST /api/registration/provider</code> |

**There is no separate admin-create-user endpoint, service or table.** The modals import from `features/auth/authApi.js`, reuse `features/auth/authValidation.js`, and hit exactly the same two routes the public sign-up forms hit. The only differences are cosmetic: single-column layout to fit the modal, and reporting success back to the admin instead of navigating to the login page.

So an admin-created Service Provider still lands as `PENDING` with a fresh `sp_verification_applications` row and still has to be approved by the Verification Admin — an admin cannot mint a pre-approved provider through this screen.

## Questions you should have a crisp answer to

**"Where is the password hashed?"** `utils/passwordUtils.js`, bcrypt with 12 salt rounds. Nowhere else — registration, provider registration and password reset all call the same `hashPassword()`. The plaintext password never reaches the

database and is never logged.

**"What actually is the 6-character token?"** A public, human-quotable user identifier — what an admin or support conversation uses instead of the internal `user_id`. `generateUserToken()` guarantees one uppercase, one lowercase and one digit, then shuffles so the guaranteed characters aren't always in the same positions. `generateUniqueToken()` retries up to 10 times against the DB, and `uix_users_user_token` plus `chk_token_format` make the database the final authority.

**"How does logging out actually invalidate a JWT?"** It doesn't invalidate the JWT itself — a signed token can't be unsigned. Each login inserts an `auth_sessions` row and embeds its `session_id` in the token as `sid`. Logout sets `revoked_at` on that row, and `checkAccountStatus` — which runs on **every** authenticated request — rejects a token whose session row is revoked. Same mechanism makes a ban take effect mid-session instead of waiting for the token to expire.

**"Why 401 and not 403 for a banned user mid-session?"** Deliberate. The frontend axios interceptor auto-logs-out on 401, so the stale token is cleared immediately. A 403 would leave the dead token sitting in storage.

**"Doesn't `forgotPassword` leak which emails are registered?"** No — that's why it `return`s silently on an unknown email instead of throwing. The response is identical either way. The token stored in `password_reset_tokens` is a SHA-256 hash; only the raw token goes out in the email, so a database read alone cannot reset anyone's password.

**"What stops a reset link being reused?"** `findValidResetToken()` requires `used_at IS NULL AND expires_at > now()`, and `resetPassword()` updates the password and marks the token used **inside one transaction**.

**"Can a rejected Service Provider just sign up again with a new email?"** Yes — that's a genuinely fresh account, and nothing in the system prevents it. What `resolveReapplicant()` handles is the *same identity* reapplying: same username and email, correct password, capped at 3 total attempts. The account is never deleted, so that username/email is permanently spent afterwards.

**"Where does the invoice amount come from?"** For a **Card** job, from `getCardPaymentAmount()` in Visal's `payment.service.js` — the provider cannot edit it, and `invoice.validation.js` documents that a submitted `amount` is ignored for Card. For a **Cash** job the provider types it, because HomeHero never handled that money and has no verified figure. That is why the invoice is explicitly not a HomeHero-verified payment record for Cash jobs.

**"Can an invoice be regenerated or edited?"** No. `invoices_booking_id_key` is a UNIQUE constraint on `booking_id`, and `generateInvoice()` checks first *and* catches Postgres error `23505` as a fallback — so even two simultaneous requests produce exactly one invoice. Download always returns the same stored PDF.

**"What happens if the PDF is written but the database insert fails?"** `fs.rmSync(absolutePath, { force: true })` in the catch block deletes the file before rethrowing. No orphaned PDF, no invoice row.

**"Who can download an invoice?"** Three parties, checked in `getInvoiceDownload()`: the owning provider, the owning client, and a System Admin (Vihas's SP Tracking). Everyone else gets 403. Note that ownership is verified against the invoice's own booking, not against anything the caller sends.

**"The System Admin can create accounts — is that a second registration implementation?"** No, and that's the point worth making. `AddClientModal` and `AddProviderModal` import `registerClient()` / `registerProvider()` from `features/auth/authApi.js` and post to the same two routes as the public forms. One registration service, one validation schema, one set of uniqueness rules — so an account created by an admin is indistinguishable from a self-registered one, and a provider created that way is still `PENDING` until Vihas approves it. Only the UI is Vihas's.

**"What happens if the welcome email fails?"** Nothing. `sendEmail()` swallows the error and logs it, and it is called **after** `COMMIT`. A mail outage can never roll back a registration.

---

## Database objects you own

---

Verified against `database/schema.sql` and the live DB ( `HomeHero_latest V10.backup` ).

## Tables (5)

| Table                 | PK                | Purpose                                                                       |
|-----------------------|-------------------|-------------------------------------------------------------------------------|
| users                 | user_id           | Every account in the system, all five roles                                   |
| roles                 | role_id           | Reference data: CLIENT , SERVICE_PROVIDER , SYSTEM_ADMIN , VERIFICATION_ADMIN |
| auth_sessions         | session_id (uuid) | One row per login; what makes logout and mid-session bans work                |
| password_reset_tokens | password_reset_id | Hashed, single-use, time-limited reset tokens                                 |
| invoices              | invoice_id        | One row per completed booking's invoice                                       |

users is the single most referenced table in the whole database — nearly every other member's tables have a foreign key into it. Worth saying out loud: **you own the table everyone depends on.**

## CHECK constraints

| Constraint            | Table    | Rule                                                                  |
|-----------------------|----------|-----------------------------------------------------------------------|
| chk_token_format      | users    | 6 alphanumeric containing at least one upper, one lower and one digit |
| invoices_amount_check | invoices | amount > 0                                                            |

## UNIQUE constraints and unique indexes — your real business rules

| Constraint / Index                   | Enforces                                                           |
|--------------------------------------|--------------------------------------------------------------------|
| uix_users_username                   | <b>One account per username</b> (and citext , so case-insensitive) |
| uix_users_email                      | <b>One account per email</b> , case-insensitive                    |
| uix_users_user_token                 | The 6-character token is globally unique                           |
| invoices_booking_id_key              | <b>One invoice per booking</b> — regeneration is impossible        |
| auth_sessions_refresh_token_hash_key | No two sessions share a refresh hash                               |
| password_reset_tokens_token_hash_key | No duplicate reset token                                           |
| roles_role_code_key                  | One row per role code                                              |

## Indexes

| Index                   | Table (column)              | Serves                                  |
|-------------------------|-----------------------------|-----------------------------------------|
| ix_users_account_status | users (account_status)      | Vihas's user management filtering       |
| ix_users_role_id        | users (role_id)             | Role-based listings                     |
| ix_users_created_at     | users (created_at)          | Registration-over-time reporting        |
| ix_inv_provider_user_id | invoices (provider_user_id) | A provider's invoice list / SP Tracking |

Lookups by username, email, token hash and booking id all ride the unique indexes above, so they need no extra index.

## Foreign keys

users → roles · auth\_sessions → users · password\_reset\_tokens → users · invoices → bookings · invoices → service\_provider\_profiles

## Views

**You own none.** You read `service_provider_profiles.verification_status` directly in `findUserByUsernameOrEmail()` rather than through `vw_latest_provider_verification`, because login needs the provider's *current* status, not the latest application's.

## Triggers, functions, stored procedures — you own none

The live database has 6 trigger functions and 6 triggers, and none are on your tables. Two of them — `trg_check_client_role` and `trg_check_provider_role` — fire on *your* insert path ( `client_profiles` / `service_provider_profiles` ), but they belong to Tharinsa's and Maheli's tables. They are a good thing to point at: they guarantee a Client profile can never be attached to a non-Client user, even if your service layer had a bug.

Be ready to say why you have none of your own, because it sounds like a gap and isn't:

*My module's invariants are enforced declaratively rather than procedurally — a CHECK constraint for the token format, unique indexes for one-account-per-username/email and one-invoice-per- booking, and multi-statement writes wrapped in explicit transactions in the service layer. Session revocation is deliberately a query on every request rather than a trigger, because it has to react to another member's table ( `user_bans` ) changing.*

**Trap to avoid:** `database/functions/` and `database/triggers/` contain **0-byte placeholder files** left over from early planning. If a panelist opens one expecting a stored procedure, say plainly that the folders were scaffolding and the logic lives in the service layer inside transactions.

## Gaps I found

- email\_logs is dead.** The table exists with a foreign key to `users`, but nothing in `server/src` ever writes to it — `email.service.js` only logs failures to the console. Either populate it in `sendEmail()` or acknowledge it as unimplemented; don't claim emails are audited.
- \* `server/src/templates/emails/ .html` are all 0 bytes.\*\* Eight placeholder files. Every email body is actually an inline template literal in `email.service.js`. Same for `emails/email.queries.js`, `emails/emailQueue.service.js` and `emails/emailTemplates.js` — all empty.
- middleware/rateLimiter.js is empty — there is no rate limiting on `/login` or `/forgot-password`.** This is the single most likely security question you will get. The honest answer: bcrypt at 12 rounds makes brute-forcing expensive, but there is no attempt cap or lockout, and adding `express-rate-limit` to those two routes is the fix.
- verificationStatus is baked into the JWT at login.** `checkAccountStatus` re-queries the DB for account status and bans, but not for verification. A provider approved *while logged in* keeps a stale `PENDING` token until they log in again. Not a security hole (it fails closed), but worth knowing before someone asks.
- auth\_sessions.ip\_address, user\_agent and last\_used\_at are never populated,** and `refresh_token_hash` is filled with a random value purely to satisfy `NOT NULL / UNIQUE` — there is no refresh-token flow. The columns are honest scaffolding for a feature that wasn't built; say so rather than inventing a purpose.
- Expired sessions and used reset tokens are never cleaned up.** Both tables grow forever. Harmless at demo scale, but a scheduled delete would be the production answer.
- An admin-created account is audited as self-registration.** Because the Add User modals reuse the public routes, `registerClient()` writes `CLIENT_REGISTERED` with `actorUserId = the new user`, and the registration routes are unauthenticated, so the backend never learns which admin created the account. Reusing one registration path is the right call; the missing piece is an optional "created by" attribution on the audit entry.

## One weak spot to be ready for

**There are no automated tests for this module** — `server/tests/` contains only `setup.js`. If asked how you verified the authentication logic, answer honestly: manual testing through the UI, including the failure paths (wrong password, banned account, expired reset link, duplicate username, a rejected provider reapplying). If you want to strengthen this

before the viva, the highest-value tests would be `generateUserToken()` (pure, trivially testable against the CHECK constraint's rules) and `resolveReapplicant()` (the branchiest logic you own).